Ellementry AI Hub — Architecture (Updated)

Sep 24, 2026 · @Rajat

An AI-powered operations intelligence platform: a shared, clean data layer feeding specialised AI agents that deliver reporting, alerts, and decision support across the business.

Solution Architecture

Four layers, designed for scalability, reliability, and maintainability:

- **AI reasoning & orchestration** — agents coordinate over a shared knowledge base so new marketplaces, business functions, or capabilities can be added without redesigning the platform.
- **Data layer** — structured business data (orders, inventory, finance, marketing) lives in PostgreSQL, modeled as a star schema. Document/text embeddings for semantic search (SOPs, meeting notes, product docs) live in the same PostgreSQL instance via the pgvector extension. Media (images, design files, CAD, packaging art) sits in object storage.
- **Ingestion** — hybrid approach: n8n manages source connectors, scheduling, retries, and notifications through its pre-built integrations, while all transformation, embedding, and business logic runs in version-controlled Python services. This keeps the intelligence centralised, testable, and independent of workflow tooling.
- **Application** — a Python (FastAPI) backend serves a React frontend dashboard; agent orchestration and business logic run as versioned services behind the API, decoupled from n8n so ingestion tooling can be swapped later without touching the platform.

Technology Stack

Layer	Technology
AI reasoning	LLM API (model choice pending)
Agent orchestration	LangGraph
Embeddings	Open embedding model, stored in PostgreSQL

Layer	Technology
Vector search	PostgreSQL + pgvector
Structured data	PostgreSQL (star schema: fact/dimension tables)
Source connectors & alert delivery	n8n pre-built integrations, scheduling, retries; business logic stays in Python services
Background jobs & queues	Redis + a task queue (e.g. Celery/RQ) for embedding runs, scheduled ingestion, report generation
Backend	Python + FastAPI
Frontend	React
Object storage	Cloud storage for images, design files, CAD, packaging art
Document & media processing	PDF/DOCX parsing, OCR, speech-to-text for meeting notes, image tagging
Alert channels	Slack API, WhatsApp Business API, email
Knowledge base	Notion + Google Drive
Authentication & access control	Role-based permissions per agent/dashboard

AI Agent Teams

Each agent has a clearly defined responsibility and shares the same underlying data model, so agents give consistent answers and new ones can be added without redesigning the platform.

Agent	Responsibility
Executive AI	Daily briefing on revenue, ROAS, orders, inventory, and product performance
Marketing AI	Monitors digital advertising: ROAS changes, performance trends, budget opportunities

Agent	Responsibility
Commerce AI	Marketplace performance, product performance, feedback, website conversion, sales bottlenecks
Product AI	Instant product information via SKU lookup
Stock AI	Inventory health, demand forecasts, replenishment recommendations, stock risks
Finance AI	EBITDA, margins, costs, cash flow, financial health
Customer AI	Consolidates product reviews and customer feedback into actionable insights
Design AI	Organises moodboards, packaging, and artwork; surfaces design and collection improvements
Operations AI	Tracks shipments, warehouse operations, returns, replacements, supply-chain performance

Prototype scope: start with two agents (to be defined) built on top of the core data model, then expand.

Data Ingestion Cadence

Frequency	Sources
Daily	ERP, shopping/marketplace sites, online ads, inventory, warehouse MIS
Weekly	Sales, finance, competitor reports, customer feedback
Monthly	Product catalog, SOPs, pricing, vendor data

Proactive alerts (threshold- and anomaly-based) notify teams of critical events — declining ROAS, low inventory, listing rejections, falling conversions, retailer issues — without needing constant dashboard monitoring.

A weekly executive report covers revenue, marketing, marketplace performance, inventory, team productivity, customer signals, key risks, priorities, and recommended actions in one summary.

Connected Data Sources

Category	Systems
Commerce & Web	eCommerce platforms (Amazon, Shopify), Website
Marketing	Google Ads, Meta Ads
Operations	ERP, Warehouse MIS, Quick commerce platforms
Knowledge & collaboration	Notion, Google Drive

Changes from the Original Proposal

- Backend: Node.js/Express → **Python + FastAPI**
- Structured data store: MongoDB → **PostgreSQL** (fits the relational shape of orders, SKUs, inventory, ad spend better; a star schema is the natural fit for BI reporting)
- Vector store: Pinecone → **PostgreSQL + pgvector** (avoids running a second database for semantic search; revisit a dedicated vector store only if scale or latency demands it)
- Frontend: React (unchanged)
- Build approach: foundation-first — the clean, modeled PostgreSQL data layer is built before any agents, since every agent in this platform reads from that same layer. First working build is a two-agent prototype on top of it, rather than all agents at once.